

D

INSTITUTO FEDERAL DO ESPÍRITO SANTO
COORDENADORIA DO CURSO DE ENGENHARIA DE CONTROLE E
AUTOMAÇÃO

MATHEUS CORTELETTI DELFINO

**ESTUDO DA UTILIZAÇÃO DO KUBERNETES PARA APLICAÇÕES
MODELADAS COM IEC 61499**

Serra
2025

MATHEUS CORTELETTI DELFINO

**ESTUDO DA UTILIZAÇÃO DO KUBERNETES PARA APLICAÇÕES
MODELADAS COM IEC 61499**

Trabalho de conclusão de curso apresentado conforme
regramento do curso de Engenharia de Controle e
Automação do Instituto Federal do Espírito Santo
como requisito para obtenção do Bacharel em
Engenharia de Controle e Automação.

Orientadora: Prof. Dr./MSc. Rafael Emerick Zape
de Oliveira

Serra
2025

RESUMO

«Apresentar brevemente o conceito principal do trabalho, como o Cluster API, e destaque o interesse recente da indústria em tecnologias de orquestração e controle distribuído.»

«Explicar os desafios que existem hoje para integrar múltiplos ambientes industriais ou computação de borda, e por que isso exige novas soluções.»

Neste contexto, este trabalho propõe uma investigação revisional sobre o uso do Cluster API como mecanismo de controle distribuído em cenários industriais com múltiplos nós de operação, como fábricas remotas ou dispositivos de borda.

«Descrever a proposta geral do trabalho, indicando que ela envolve tanto um levantamento bibliográfico estruturado quanto uma validação prática em um ambiente experimental.»

Para isso, foi construído um ambiente com um nó de controle e múltiplos nós gerenciados em diferentes localizações, permitindo avaliar aspectos como latência, confiabilidade e viabilidade de adoção em ambientes industriais reais.

«Resumir as técnicas, ferramentas ou abordagens usadas — ex: tipos de clusters testados, uso de alguma nuvem pública, métricas medidas etc.»

Os testes realizados demonstraram «insira aqui uma síntese dos principais resultados esperados ou obtidos».

Os resultados sugerem que o uso do Cluster API pode representar uma solução promissora para a integração e controle de sistemas industriais distribuídos, especialmente em contextos que exigem escalabilidade, autonomia e baixa latência.

Palavras-chave: Cluster API. Computação de borda. Orquestração de clusters. Indústria 4.0. Integração distribuída.

ABSTRACT

«aguardando fechar o resumo»

Keywords: «aguardando fechar o resumo»

LISTA DE FIGURAS

LISTA DE TABELAS

SUMÁRIO

1 INTRODUÇÃO

A transformação digital na indústria, impulsionada pelo conceito de Indústria 4.0, vem redefinindo sistemas de produção ao integrar tecnologias avançadas de informação e comunicação aos processos industriais. Essa quarta revolução industrial caracteriza-se pela incorporação de sensores inteligentes, computação em nuvem, big data, inteligência artificial, robótica e outras inovações às fábricas, com o objetivo de criar operações mais conectadas, automatizadas e inteligentes. Conceitos como manufatura inteligente e produção orientada por dados tornaram-se centrais, visando aumentar a eficiência, reduzir custos e melhorar a qualidade de produtos e processos (????). As empresas têm ampliado o uso de sistemas baseados em Internet das Coisas Industrial (IIoT) como parte desse movimento, buscando maior confiabilidade e competitividade por meio da conectividade ubíqua de dispositivos e da análise de dados em tempo real (??).

Implementar essa visão, contudo, traz desafios significativos, especialmente no que tange à arquitetura de automação industrial. A integração de múltiplas tecnologias heterogêneas (equipamentos legados, redes industriais, plataformas de TI) exige alto grau de interoperabilidade e flexibilidade. Isso implica novos requisitos para os sistemas de controle industrial, como maior descentralização, modularidade e autonomia, características que as arquiteturas tradicionais hierárquicas não conseguem suprir completamente (??). Em outras palavras, há uma demanda crescente por controle distribuído, no qual a tomada de decisão e a execução do controle estejam espalhadas por diversos dispositivos inteligentes na planta, em vez de concentradas em um único controlador central. Atender a essa demanda requer repensar modelos de desenvolvimento de software industrial e adotar padrões que suportem tal distribuição de forma padronizada e reconfigurável (??).

Uma das respostas a essa necessidade foi a criação do padrão IEC 61499, desenvolvido pela Comissão Eletrotécnica Internacional para prover um modelo de referência de sistemas de controle distribuído baseado em blocos funcionais. O IEC 61499 define uma linguagem de modelagem específica de domínio para desenvolvimento de aplicações de controle distribuído, estendendo o padrão IEC 61131-3 (utilizado em CLPs convencionais) com melhor encapsulamento de componentes de software, formato independente de fornecedor e simplificação da comunicação controlador-a-controlador (????). Em essência, suas funcionalidades de distribuição e suporte à reconfiguração dinâmica fornecem a infraestrutura necessária para viabilizar aplicações industriais inteligentes conforme preconiza a Indústria 4.0 e a IIoT (????). Assim, arquiteturas baseadas em IEC 61499 permitem conceber sistemas nos quais múltiplos dispositivos executam, de forma coordenada, partes de uma mesma aplicação de controle, contribuindo para uma manufatura mais flexível e adaptável.

Paralelamente a essas evoluções na camada de controle, o domínio de Tecnologia da

Informação (TI) tem assistido ao amadurecimento de ferramentas de virtualização e gerenciamento de aplicativos em larga escala. Em especial, a containerização de aplicações (por meio de plataformas como Docker) e as plataformas de orquestração de contêineres se tornaram o padrão em infraestruturas de TI. O uso de contêineres permite empacotar softwares com todas as suas dependências, garantindo portabilidade e isolamento, enquanto orquestradores automatizam a implantação e escalonamento desses contêineres em clusters de servidores (????). Essa tendência agora avança em direção ao chão de fábrica: a virtualização de aplicações e sua orquestração em dispositivos de borda é um fenômeno recente no contexto de IIoT/Indústria 4.0 (??). Ferramentas baseadas em Kubernetes, em particular, despontam como candidatas naturais para gerenciar aplicações industriais containerizadas distribuídas, dadas suas capacidades robustas de automação e escalabilidade (??). O Kubernetes já se firmou como o padrão em orquestração de contêineres na indústria de software, oferecendo um rico conjunto de funções para implantar, monitorar, escalar e recuperar aplicações containerizadas de forma declarativa. Estudos indicam inclusive que casos de uso de borda na manufatura vêm adotando amplamente o Kubernetes – uma pesquisa da CNCF mostrou que grande parte dos projetos de edge computing industrial utiliza variantes do Kubernetes (??).

Frente a esse panorama, o presente trabalho propõe avaliar o uso da Cluster API – uma extensão do Kubernetes voltada ao gerenciamento declarativo de clusters – como ferramenta de orquestração de aplicações de controle distribuído modeladas segundo o padrão IEC 61499 e empacotadas em contêineres Docker. Em termos práticos, será desenvolvida uma prova de conceito na qual uma aplicação industrial baseada em blocos funcionais IEC 61499 é distribuída entre diferentes nós computacionais, executando dentro de contêineres e gerenciada de forma unificada por um cluster Kubernetes. A Cluster API será empregada para automatizar a configuração e coordenação desse cluster, permitindo, por exemplo, provisionar e configurar múltiplos nós de forma consistente e coordenada (??). O foco da pesquisa reside na avaliação de desempenho dessa arquitetura proposta: mediremos métricas como latência de comunicação entre blocos de controle distribuídos, sobrecarga introduzida pela camada de orquestração, tempo de resposta a eventos de escalonamento, entre outras. Espera-se, com isso, verificar a viabilidade e os potenciais gargalos de aplicar tecnologias nativas de nuvem (como Kubernetes e Cluster API) em cenários de automação industrial distribuída sob a ótica do IEC 61499 (????).

1.1 PROBLEMA

Aplicações industriais distribuídas, especialmente aquelas modeladas segundo o padrão IEC 61499, impõem uma série de desafios para implantação, gerenciamento e escalonamento. Controladores distribuídos exigem coordenação precisa, alta disponibilidade, sincronização entre dispositivos, suporte a reconfiguração dinâmica e operação em borda com restrições

de tempo real. No entanto, a orquestração dessas aplicações ainda carece de soluções padronizadas, portáteis e automatizadas, dificultando sua adoção em larga escala.

1.2 A PROPOSTA

Este trabalho propõe utilizar a Cluster API — uma extensão do ecossistema Kubernetes — como mecanismo de orquestração para aplicações industriais distribuídas baseadas no padrão IEC 61499 e empacotadas em contêineres Docker. A proposta será validada por meio do desenvolvimento de uma prova de conceito que distribui uma aplicação IEC 61499 entre diferentes nós, executando em contêineres e gerenciada de forma declarativa via Cluster API.

1.3 OBJETIVOS

1.3.1 Objetivo Geral

Avaliar o uso da Cluster API como ferramenta de orquestração de aplicações industriais de controle distribuído modeladas com IEC 61499.

1.3.2 Objetivos Específicos

1. Configurar um ambiente experimental com suporte a Cluster API, Kubernetes e contêineres Docker.
2. Modelar uma aplicação de controle distribuído com IEC 61499, adaptada para execução containerizada.
3. Integrar a aplicação ao ambiente orquestrado pela Cluster API.
4. Medir métricas de desempenho como latência, tempo de resposta a falhas e sobrecarga da orquestração.
5. Analisar a viabilidade e os gargalos da abordagem proposta em cenários industriais reais.

1.4 ORGANIZAÇÃO DO TRABALHO

Estrutura do trabalho: Este documento está organizado da seguinte forma. No Capítulo 2, são revisados os principais conceitos e fundamentos teóricos que embasam o estudo, incluindo Indústria 4.0, IIoT, controle distribuído com IEC 61499, containerização, microserviços, Kubernetes e Cluster API, além de uma discussão sobre desafios de

interoperabilidade, latência e escalabilidade em sistemas de controle distribuído. Em seguida, o Capítulo 3 descreverá a metodologia adotada e o desenvolvimento da prova de conceito, detalhando o ambiente experimental. O Capítulo 4 apresentará os resultados obtidos e uma análise crítica do desempenho observado. Por fim, no Capítulo 5, são expostas as conclusões do trabalho e sugestões de trabalhos futuros relacionados.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que embasam o desenvolvimento deste trabalho. São abordados o Processo Tennessee Eastman como benchmark industrial, o padrão IEC 61499 para controle distribuído, o padrão de Operators do Kubernetes como mecanismo de orquestração, o protocolo gRPC como canal de comunicação, a integração numérica por Runge-Kutta de 4ª ordem como método de simulação, e o conceito de gêmeo digital como paradigma que unifica os demais.

2.1 Processo Tennessee Eastman (TEP)

O Processo Tennessee Eastman (*Tennessee Eastman Process* — TEP) é um *benchmark* amplamente utilizado na comunidade de controle de processos para avaliação de estratégias de controle, detecção de falhas e otimização de operações industriais. Proposto originalmente por Downs e Vogel em 1993, o TEP descreve uma planta química realista com dinâmica não-linear complexa, laços de reciclo e múltiplas perturbações programadas, sendo deliberadamente desafiador do ponto de vista de controle (??).

A planta consiste em cinco unidades de processo interconectadas: um reator, um condensador, um separador de vapor-líquido (*flash separator*), um compressor e uma coluna de *stripping*. Quatro reagentes (A, B, C, D) são alimentados ao reator, onde ocorrem três reações:



O componente B é inerte e não participa das reações. Os produtos G e H são os produtos desejados; F é um subproduto indesejado. As reações são exotérmicas, e a temperatura do reator é controlada indiretamente pela água de resfriamento (*cooling water* — CW).

O modelo matemático é descrito por um sistema de 50 equações diferenciais ordinárias (EDO), representando os holdups de massa e energia de cada vaso de processo. O vetor de estado $\mathbf{y} \in R^{50}$ inclui os inventários de cada componente nos reatores, separadores e no espaço de vapor, bem como as energias internas de cada subsistema. O modelo completo foi originalmente implementado em FORTRAN e posteriormente portado para diversas linguagens, mantendo a estrutura matemática original.

O modelo expõe 41 variáveis medidas (XMEAS₁ a XMEAS₄₁) e 12 variáveis manipuladas (XMV₁ a XMV₁₂). As variáveis medidas incluem vazões, composições, temperaturas, pressões e níveis dos principais subsistemas; as variáveis manipuladas correspondem a

posições de válvulas e setpoints de controladores de baixo nível. O ponto de operação nominal (Modo 1) é caracterizado por pressão do reator de 2705 kPa, temperatura do reator de 120,4 °C e frações molares de produto (razão G/H) específicas (??).

O modelo inclui 20 distúrbios programados (IDV_1 a IDV_{20}) que simulam eventos operacionais como variações de composição de *feed*, falhas em trocadores de calor e contaminações. Cada distúrbio atua diretamente sobre parâmetros do modelo, permitindo avaliar a robustez de estratégias de controle frente a perturbações realistas. O modelo também define condições de parada por segurança (*Interlock Shutdown Device* — ISD), que encerram a simulação quando variáveis críticas ultrapassam limites operacionais (por exemplo, pressão do reator acima de 3000 kPa ou nível do reator abaixo de 10%).

A relevância do TEP como *benchmark* decorre de sua riqueza dinâmica: o modelo combina não-linearidades, acoplamentos entre malhas de controle, laços de reciclo com elevada constante de tempo e perturbações com mecanismos físicos distintos. Essas características tornam o TEP um ambiente adequado para avaliar arquiteturas de controle supervisorio — em especial aquelas que dependem de comunicação distribuída e coordenação entre componentes heterogêneos, como a arquitetura proposta neste trabalho.

2.2 Controle Distribuído e IEC 61499

A evolução dos sistemas de automação industrial em direção a arquiteturas mais flexíveis e descentralizadas impulsionou o desenvolvimento do padrão IEC 61499, publicado pela Comissão Eletrotécnica Internacional como resposta às limitações do IEC 61131-3 em ambientes distribuídos (??). Enquanto o IEC 61131-3 organiza a execução em dispositivos centralizados e utiliza execução síncrona por varredura (*scan*), o IEC 61499 define um modelo orientado a eventos, distribuível entre múltiplos dispositivos e reconfigurável em tempo de execução (??).

O elemento fundamental do IEC 61499 é o **Bloco Funcional** (*Function Block* — FB). Um FB encapsula dados e algoritmos em um componente com duas interfaces distintas: uma interface de eventos (entradas e saídas de eventos que disparam e notificam execuções) e uma interface de dados (entradas e saídas de dados tipados). A execução de um FB é disparada por eventos — não por varredura temporal — o que permite modelar sistemas reativos e assíncronos de forma natural (??).

O padrão define três tipos de FBs:

1. **FB Básico** (*Basic FB*): contém uma máquina de estados de execução (*Execution Control Chart* — ECC) e algoritmos em linguagens de IEC 61131-3. É a unidade atômica de computação.

2. **FB Composto** (*Composite FB*): encapsula uma rede de outros FBs, permitindo composição hierárquica.
3. **FB de Interface de Serviço** (*Service Interface FB* — SIFB): abstrai comunicação com hardware ou serviços externos, como protocolos de rede e E/S de campo.

O modelo de aplicação do IEC 61499 descreve redes de FBs interconectados por seus portos de eventos e dados, formando o grafo de controle da aplicação. O modelo de dispositivo define como essa aplicação é particionada em recursos computacionais (*resources*) distribuídos por múltiplos dispositivos físicos. O mapeamento da aplicação aos dispositivos define a distribuição, e a comunicação entre FBs em dispositivos diferentes é tratada pelos SIFBs de comunicação — preservando a transparência de localização do ponto de vista do desenvolvedor (??).

A capacidade de reconfiguração dinâmica é um diferencial significativo do IEC 61499: é possível adicionar, remover ou reconfigurar FBs em tempo de execução sem parar a aplicação, o que permite adaptação a mudanças operacionais sem intervenção manual extensa (??). Essa característica alinha-se diretamente com os objetivos de Indústria 4.0 de sistemas de manufatura reconfigurável e autoadaptável.

Do ponto de vista do presente trabalho, o TEP implementado em contêineres Docker pode ser concebido como uma aplicação IEC 61499 distribuída: cada controlador (*P-controller*) corresponde a um FB básico; o **ControllerBank** corresponde a um FB composto que orquestra os FBs individuais; e o CRD **PLCMachine** no Kubernetes define declarativamente a política de controle — análogo ao modelo de dispositivo do IEC 61499 que descreve como a aplicação deve ser implantada e operada. O Operator Kubernetes assume o papel do runtime de implantação e reconfiguração (??).

2.3 Kubernetes e o Padrão de Operators

O Kubernetes é uma plataforma de orquestração de contêineres de código aberto desenvolvida originalmente pelo Google e atualmente mantida pela *Cloud Native Computing Foundation* (CNCF). Sua arquitetura é centrada no princípio da **reconciliação declarativa**: o usuário descreve o estado desejado do sistema em manifestos YAML, e o plano de controle do Kubernetes trabalha continuamente para fazer o estado real convergir ao estado desejado (??).

A arquitetura do Kubernetes divide-se em plano de controle (*control plane*) e nós de trabalho (*worker nodes*). O plano de controle inclui o *API Server* (único ponto de entrada para todas as operações), o *etcd* (banco de dados distribuído de chave-valor que armazena o estado do cluster), o *Scheduler* (alocador de cargas de trabalho) e o *Controller Manager*

(gerenciador dos controladores nativos). Os nós de trabalho executam os contêineres via *kubelet* e *container runtime*.

O Kubernetes permite estender seu modelo de recursos por meio das **Definições de Recursos Customizados** (*Custom Resource Definitions* — CRDs). Um CRD declara um novo tipo de recurso no API Server, com seu próprio esquema (*spec* e *status*), tornando-o disponível para criação, listagem e edição com as mesmas ferramentas (*kubectl*, YAML) usadas para recursos nativos. A separação entre *spec* (intenção declarada pelo usuário) e *status* (estado observado pelo sistema) segue o mesmo princípio dos recursos nativos e é fundamental para o padrão de Operator (??).

O **padrão de Operator** (*Operator Pattern*) é um método de extensão do Kubernetes que codifica o conhecimento operacional de uma aplicação específica em um controlador customizado (??). Um Operator combina um ou mais CRDs com um *controller loop* que observa instâncias desses recursos e age para reconciliar o estado real com o estado desejado — substituindo a intervenção humana pela automação do domínio de conhecimento específico da aplicação.

O *loop* de reconciliação de um Operator segue o padrão **Observar–Avaliar–Agir**:

1. **Observar**: o controller recebe uma notificação de que um recurso customizado foi criado, modificado ou deletado (via *watch* no API Server).
2. **Avaliar**: o controller lê o estado atual da aplicação (possivelmente via chamadas externas, como gRPC) e compara com o estado desejado descrito no *spec* do CRD.
3. **Agir**: se houver divergência, o controller toma ações corretivas — no caso deste trabalho, chamadas gRPC à planta para ajustar parâmetros de controle.

O framework **Kubebuilder** é a ferramenta oficial da CNCF para construção de Operators em Go. Ele provê scaffolding de projeto, integração com o *controller-runtime* (biblioteca Go para interação com o API Server), geração de CRDs a partir de structs Go anotadas, e utilitários para testes (??). A anotação dos campos do struct com marcadores como `+kubebuilder:validation` permite validação automática de manifests, e `+kubebuilder:subresource` habilita o subrecurso de *status* separado da *spec*.

Para operações industriais, o padrão de Operator oferece vantagens relevantes: a lógica supervisória é codificada uma vez e executada de forma autônoma; a política de operação é declarativa e auditável em repositórios de versionamento; e o Kubernetes garante alta disponibilidade do próprio operator via restarts automáticos (??).

2.4 gRPC como Protocolo de Comunicação

O gRPC é um *framework* de chamada de procedimento remoto (*Remote Procedure Call* — RPC) de alto desempenho desenvolvido pelo Google, baseado no protocolo HTTP/2 e no sistema de serialização *Protocol Buffers* (protobuf) (??). Diferentemente de alternativas baseadas em REST/JSON, o gRPC utiliza serialização binária tipada, o que reduz o tamanho das mensagens e o tempo de serialização — vantagem relevante para aplicações de automação que transmitem dados de sensores em alta frequência.

A definição de um serviço gRPC é feita em um arquivo `.proto`, que especifica os métodos RPC e os tipos de mensagem de forma agnóstica à linguagem. A partir desse arquivo, o compilador `protoc` gera *stubs* de cliente e servidor para as linguagens desejadas — no caso deste trabalho, Rust (biblioteca *tonic*) para a planta, Go (biblioteca *google.golang.org/grpc*) para o operador, e Python (biblioteca *grpcio*) para a IHM. A consistência do contrato é garantida pelo arquivo `.proto` canônico, mantido no repositório `tep-plant`.

O gRPC suporta quatro modalidades de comunicação:

1. **Unário:** uma requisição e uma resposta — usado para comandos pontuais (e.g., `GetPlantStatus`, `UpdateController`).
2. **Server streaming:** uma requisição e um fluxo contínuo de respostas — usado para streaming de métricas (`StreamMetrics`).
3. **Client streaming:** fluxo de requisições e uma resposta.
4. **Bidirecional:** fluxos simultâneos em ambas as direções.

No contexto industrial, o *server streaming* é particularmente adequado para a transmissão contínua de dados de processo: a IHM abre um único fluxo gRPC e recebe atualizações das 41 XMEAS e 12 XMV a cada passo de simulação, sem *polling* periódico. O protocolo HTTP/2 subjacente permite múltiplos fluxos simultâneos em uma única conexão TCP, reduzindo overhead de estabelecimento de conexão (??).

A escolha do gRPC como protocolo de comunicação neste trabalho é motivada por três fatores: (a) suporte nativo a *streaming* de alta frequência com baixa latência; (b) contrato fortemente tipado que detecta incompatibilidades entre componentes em tempo de compilação; e (c) suporte multiplataforma (Rust, Go, Python) sem adaptadores intermediários, permitindo uma arquitetura poliglota coesa.

2.5 Integração Numérica por Runge-Kutta de 4ª Ordem

A simulação dinâmica do TEP requer a integração numérica de um sistema de 50 EDOs da forma:

$$\frac{d\mathbf{y}}{dt} = f(\mathbf{y}, \mathbf{u}, t) \quad (4)$$

onde $\mathbf{y} \in R^{50}$ é o vetor de estado, $\mathbf{u} \in R^{12}$ é o vetor de variáveis manipuladas e f é a função de dinâmica da planta — altamente não-linear, com acoplamentos termodinâmicos e de transferência de massa.

O método de **Runge-Kutta de 4ª ordem** (RK4) é o integrador clássico para EDOs de valores iniciais com dinâmica suave. Para um passo de tempo h , a atualização do estado é dada por:

$$\begin{aligned} k_1 &= h \cdot f(\mathbf{y}_n, t_n) \\ k_2 &= h \cdot f\left(\mathbf{y}_n + \frac{k_1}{2}, t_n + \frac{h}{2}\right) \\ k_3 &= h \cdot f\left(\mathbf{y}_n + \frac{k_2}{2}, t_n + \frac{h}{2}\right) \\ k_4 &= h \cdot f(\mathbf{y}_n + k_3, t_n + h) \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned} \quad (5)$$

O RK4 possui erro local de truncamento de ordem $\mathcal{O}(h^5)$ e erro global de ordem $\mathcal{O}(h^4)$, o que o torna preciso o suficiente para simulação de processos industriais com passo de tempo adequado. Para o TEP, o modelo original de Downs e Vogel especifica $dt = 0,001$ h como passo de integração padrão, valor que garante estabilidade numérica para a dinâmica mais rápida do sistema (tipicamente associada ao espaço de vapor do compressor) (??).

A norma dos derivativos $\|\mathbf{f}(\mathbf{y}, t)\|_\infty$ — denominada `deriv_norm` na implementação deste trabalho — serve como indicador de saúde numérica da simulação: valores excessivamente altos indicam instabilidade iminente ou condições iniciais inconsistentes; valores estabilizados em patamar baixo indicam operação próxima ao estado estacionário. Esse indicador revelou-se útil para detectar o impacto de distúrbios antes que as variáveis medidas apresentassem desvios visíveis.

2.6 Gêmeo Digital Industrial

O conceito de **gêmeo digital** (*digital twin*) refere-se a uma representação digital de um ativo físico, processo ou sistema que espelha seu comportamento em tempo real e

permite simulação, análise e otimização sem intervenção no ativo real (??). Em contextos industriais, gêmeos digitais são aplicados para monitoramento de saúde de equipamentos, otimização de processos, treinamento de operadores e validação de estratégias de controle antes de sua implantação.

Uma taxonomia comum classifica os gêmeos digitais em três níveis de fidelidade:

1. **Modelo digital:** representação estática ou off-line, sem sincronização em tempo real.
2. **Digital shadow:** representação que recebe dados do ativo físico mas não envia comandos de volta.
3. **Gêmeo digital completo:** integração bidirecional — recebe dados do físico e pode enviar atuações, formando um *cyber-physical loop*.

O laboratório desenvolvido neste trabalho implementa um gêmeo digital do tipo *shadow/completo*: a planta simulada em Rust replica a dinâmica do TEP (modelo digital de alta fidelidade), e o Operator Kubernetes atua sobre ela em um laço de controle supervísório — análogo ao que seria feito com um ativo físico real. Embora não haja uma planta física correspondente, a arquitetura é deliberadamente construída para que a substituição da planta simulada por dados reais requeira apenas a troca do endereço gRPC de destino, preservando toda a camada supervísória.

2.7 Considerações Finais

A revisão apresentada fundamenta as escolhas tecnológicas do trabalho: o TEP provê o modelo de processo com complexidade industrial; o IEC 61499 fornece o referencial conceitual para controle distribuído baseado em eventos; o padrão de Operator do Kubernetes realiza esse controle na prática como um controlador declarativo e autônomo; o gRPC garante comunicação eficiente e fortemente tipada entre os componentes; o RK4 integra as EDOs do modelo com precisão adequada; e o paradigma de gêmeo digital enquadra o sistema como uma arquitetura válida para investigação de orquestração industrial. Os capítulos seguintes descrevem como esses elementos foram combinados na prova de conceito.

3 DESENVOLVIMENTO

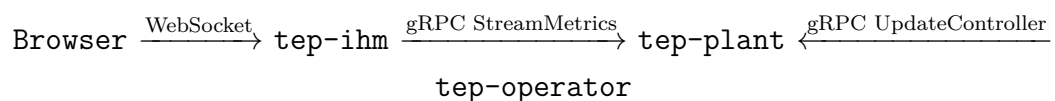
Este capítulo descreve a arquitetura proposta e os componentes desenvolvidos para implementar o laboratório de gêmeo digital do Processo Tennessee Eastman com supervisão baseada em Kubernetes. A prova de conceito é composta por quatro repositórios com responsabilidades bem delimitadas: **tep-plant** (simulador da planta), **tep-operator** (Operator Kubernetes), **tep-ihm** (Interface Homem-Máquina) e **tep-supervisor** (infraestrutura local).

3.1 Visão Geral da Arquitetura Proposta

A arquitetura do laboratório segue um modelo em três camadas, inspirado na separação entre campo, supervisão e gerenciamento presente em pirâmides de automação industrial:

1. **Camada de campo:** **tep-plant** simula a dinâmica da planta química e expõe seus estados via gRPC.
2. **Camada de supervisão:** **tep-operator** observa o estado da planta e aplica ações corretivas de acordo com uma política declarativa descrita em um CRD Kubernetes.
3. **Camada de monitoramento:** **tep-ihm** consome o fluxo de dados da planta e os apresenta ao operador humano via dashboard web em tempo real.

O fluxo de dados é o seguinte:



A planta avança o tempo simulado via integração RK4 ($dt = 0,001$ h/passos) e publica continuamente o vetor de medidas. O operador lê essas medidas a cada ciclo de reconciliação, avalia se as variáveis de processo estão dentro das faixas declaradas no CRD **PLCMachine**, e chama **UpdateController** quando necessário para ajustar setpoints ou ganhos dos controladores. A IHM consome o mesmo fluxo gRPC e transmite as medidas ao navegador via WebSocket, permitindo visualização em tempo real sem intervenção no laço de controle.

Inserir figura: diagrama de arquitetura com os quatro componentes, conexões gRPC/WebSocket e camadas.

A separação de responsabilidades é deliberada e oferece vantagens arquiteturais: o simulador não conhece o operador nem a IHM; o operador não conhece a IHM; a comunicação é

unidirecional exceto pelo canal de controle (`UpdateController`). Isso permite substituir qualquer componente — por exemplo, trocar o simulador Rust por dados de uma planta física — sem alterar os demais.

3.2 Simulador da Planta — `tep-plant`

O repositório `tep-plant` implementa o modelo matemático do TEP em Rust e expõe um servidor gRPC na porta 50051. A escolha de Rust é motivada por sua segurança de memória sem *garbage collector*, desempenho próximo ao de C/C++ e sistema de tipos expressivo — características relevantes para simulação numérica de longa duração.

3.2.1 Estrutura Interna

O repositório é organizado em dois *crates*:

- **te-core**: biblioteca que implementa o modelo matemático do TEP. Contém o vetor de estado $\mathbf{y} \in R^{50}$, a função de dinâmica `tep_derivatives()` (equivalente à subrotina TEFUNC do FORTRAN original), a lógica de verificação ISD e o integrador RK4.
- **tennessee-eastman-service**: binário que instancia o modelo, executa o loop de simulação, inicia os controladores via `ControllerBank` e serve as requisições gRPC utilizando a biblioteca *tonic*.

3.2.2 Loop de Simulação

O loop de simulação é executado em uma *thread* dedicada. A cada iteração:

1. O módulo `runtime.rs` chama `plant.step(dt)`, que internamente executa um passo RK4 com $dt = 0,001$ h.
2. Após a integração, o `ControllerBank` avalia os controladores ativos via `bank.step(&xmeas, &mut xmv)`, calculando as novas variáveis manipuladas para o próximo passo.
3. As medidas atualizadas são publicadas em um canal *broadcast* consumido pelos clientes gRPC conectados.
4. O estado completo pode ser serializado em um arquivo TOML (*snapshot*) para reinicialização futura sem *cold start*.

O tempo simulado avança a $100\times$ a velocidade real (1 h simulada por 36 s de relógio), controlado pela variável de ambiente `STEP_DELAY_MS=36`. Isso permite realizar experimentos de 20 h simuladas em aproximadamente 12 minutos de relógio.

3.2.3 Controladores — `ControllerBank`

A camada de controle é baseada na trait `Controller`:

Listing 3.1 – Trait `Controller` e `ControllerBank` (tep-plant)

```

1 pub trait Controller: Send {
2     fn step(&mut self, xmeas: &[f64], xmv: &mut [f64]);
3 }
4
5 pub struct ControllerBank {
6     controllers: Vec<Box<dyn Controller>>,
7 }
8 impl ControllerBank {
9     pub fn step(&mut self, xmeas: &[f64], xmv: &mut [f64]) {
10         for ctrl in &mut self.controllers {
11             ctrl.step(xmeas, xmv);
12         }
13     }
14 }
```

Essa abstração separa o modelo da planta da lógica de controle, tornando o `runtime.rs` agnóstico ao tipo e quantidade de controladores. A implementação concreta `PController` realiza controle proporcional:

Listing 3.2 – Controlador Proporcional (tep-plant)

```

1 pub struct PController {
2     pub xmeas_idx: usize,
3     pub xmv_idx:   usize,
4     pub kp:        f64,
5     pub setpoint:  f64,
6     pub bias:      f64,
7 }
8 impl Controller for PController {
9     fn step(&mut self, xmeas: &[f64], xmv: &mut [f64]) {
10         let error = xmeas[self.xmeas_idx] - self.setpoint;
11         xmv[self.xmv_idx] = (self.bias + self.kp * error).clamp(0.0, 100.0);
12     }
13 }
```

No ponto de operação nominal, três malhas P são instanciadas em `main.rs`:

1. **Pressão do reator** ($\text{XMEAS}_7 \rightarrow \text{XMV}_6$): $K_p = 0,10$, setpoint = 2705 kPa, bias = 40,06%.
2. **Nível do separador** ($\text{XMEAS}_{12} \rightarrow \text{XMV}_7$): $K_p = 1,00$, setpoint = 50%, bias = 38,10%.
3. **Nível do stripper** ($\text{XMEAS}_{15} \rightarrow \text{XMV}_8$): $K_p = 1,00$, setpoint = 50%, bias = 46,50%.

3.2.4 Servidor gRPC

O servidor gRPC expõe quatro métodos definidos no arquivo `tep.proto`:

- **StreamMetrics**: fluxo contínuo (*server streaming*) de XMEAS_{1-41} e XMV_{1-12} .
- **GetPlantStatus**: snapshot pontual do estado da planta.
- **ListControllers**: lista os controladores ativos e seus parâmetros.
- **UpdateController**: atualiza o setpoint ou ganho de um controlador pelo índice.

O método `UpdateController` é o canal pelo qual o operador Kubernetes age sobre a planta, fechando o laço supervisor.

3.2.5 Sistema de Distúrbios

Os distúrbios (IDV_1 a IDV_{20}) são ativados via variável de ambiente `ACTIVE_IDV` ou dinamicamente via painel da IHM. Cada distúrbio é implementado como um modificador das condições de entrada da planta, fiel ao FORTRAN original de Downs e Vogel (??). A seleção dos distúrbios relevantes para o supervisor é discutida no Capítulo ??.

3.3 Operator Kubernetes — `tep-operator`

O repositório `tep-operator` implementa um Operator Kubernetes em Go, construído com o framework Kubebuilder. Seu objetivo é atuar como supervisor de alto nível da planta TEP, avaliando o estado das variáveis de processo e aplicando ações corretivas de acordo com a política declarada no CRD `PLCMachine`.

3.3.1 O CRD `PLCMachine`

A `PLCMachine` é o recurso customizado que encapsula a política supervisória. Sua *spec* define:

- **plantAddress**: endereço gRPC da planta (e.g., `host.docker.internal:50051`).
- **variables**: lista de variáveis a monitorar, cada uma com nome, índice XMEAS, e faixas mínima e máxima aceitáveis.
- **controllers**: lista de controladores a gerenciar, com índice, setpoint nominal e ação corretiva.

Seu *status* armazena:

- **phase**: estado operacional (`Pending`, `Stable`, `Alarm`).
- **plantTime**: tempo simulado atual da planta.
- **lastReconcile**: timestamp do último ciclo de reconciliação.
- Estado observado de cada variável monitorada.

Um exemplo de manifest `PLCMachine` para o baseline TEP é apresentado no Código ??:

Listing 3.3 – Manifest `PLCMachine` para o baseline TEP

```

1  apiVersion: infrastructure.tep.io/v1alpha1
2  kind: PLCMachine
3  metadata:
4    name: tep-baseline
5  spec:
6    plantAddress: "host.docker.internal:50051"
7    variables:
8      - name: reactor_pressure
9        xmeasIndex: 7
10       min: 2600.0
11       max: 2900.0
12      - name: separator_level
13        xmeasIndex: 12
14        min: 30.0
15        max: 70.0
16      - name: stripper_level
17        xmeasIndex: 15
18        min: 30.0
19        max: 70.0

```

3.3.2 Loop de Reconciliação

O controller Go implementa o método `Reconcile(ctx, req)` chamado pelo *controller-runtime* sempre que uma `PLCMachine` é criada ou modificada. O loop de reconciliação executa as seguintes etapas:

1. **Leitura do recurso:** o controller busca a instância `PLCMachine` no API Server.
2. **Conexão com a planta:** estabelece ou reutiliza a conexão gRPC com o endereço declarado em `spec.plantAddress`.
3. **Observação:** chama `GetPlantStatus` para ler as 41 XMEAS e 12 XMV atuais.
4. **Avaliação de política:** compara cada variável monitorada com seus limites declarados no CRD.
5. **Ação corretiva:** se alguma variável estiver fora da faixa, calcula a ação de controle e chama `UpdateController`.
6. **Atualização de status:** grava o estado observado em `PLCMachine.status`.

O controller é reenfileirado automaticamente após cada ciclo, garantindo observação contínua da planta enquanto a `PLCMachine` existir no cluster.

3.3.3 Conectividade com a Planta

A planta (`tep-plant`) executa como contêiner Docker no host, enquanto o operator executa dentro do cluster Kind. Devido ao isolamento de rede do Kind, o endereço interno do Docker Compose (`te-plant:50051`) não é acessível de dentro do cluster. A solução é o endereço especial `host.docker.internal:50051`, que roteia para a interface do host Docker e alcança a planta, independentemente de onde o Kind está executando. Essa distinção é documentada como regra arquitetural do laboratório e deve ser respeitada em todos os manifests de `PLCMachine`.

3.4 Interface Homem-Máquina — `tep-ihm`

O repositório `tep-ihm` implementa a IHM do laboratório em Python, usando FastAPI como servidor web e WebSocket para comunicação em tempo real com o navegador.

3.4.1 Arquitetura

A IHM segue uma arquitetura assíncrona baseada em tarefas corrotinas (*asyncio*):

1. Uma tarefa `grpc_reader` consome o fluxo `StreamMetrics` da planta via gRPC e publica cada mensagem em uma fila assíncrona (`asyncio.Queue`).
2. Uma tarefa `ws_broadcaster` retira mensagens da fila, serializa para JSON e transmite via WebSocket a todos os clientes conectados.
3. O servidor FastAPI serve a interface HTML/JavaScript estática e gerencia as conexões WebSocket.

Essa separação garante que o consumo gRPC seja independente do número de clientes WebSocket conectados, evitando que atrasos na rede do cliente afetem o laço de coleta de dados.

3.4.2 Dashboard

A interface web do dashboard foi desenvolvida seguindo as diretrizes da norma ISA-101 para interfaces de processo, priorizando clareza informacional sobre estética. O dashboard é composto por:

- **Painel SVG da planta:** representação esquemática dos cinco subsistemas do TEP com indicação visual dos estados de alarme.
- **Gráficos de tendência:** séries temporais das principais variáveis ($XMEAS_7$ pressão, $XMEAS_9$ temperatura, $XMEAS_{12}$ nível separador, $XMEAS_{15}$ nível stripper) renderizadas via ECharts.
- **Painel de controladores:** valores atuais de cada XMV com indicação de setpoint.
- **Painel de distúrbios:** botões de ativação/desativação dos IDVs, com estado visual.
- **Painel analítico:** gráficos de composição ($XMEAS_{23-28}$) e inventários de vapor (UCVR, UCVV).
- **Status do PLCMachine:** exibe o estado atual do CRD via chamada ao API Server do Kubernetes (fase, última reconciliação, variáveis monitoradas).

Inserir figura: captura de tela do dashboard da IHM com os painéis principais.

3.4.3 Registro de Dados (CSV)

A IHM suporta gravação de todos os dados recebidos em arquivo CSV, ativada pela variável de ambiente `RECORD_CSV=true`. O arquivo é escrito continuamente durante a sessão e pode ser exportado pelo operador via botão na interface. Os CSVs gerados são a principal fonte de dados para as análises dos experimentos descritos no Capítulo ??.

Cada linha do CSV contém o timestamp, o tempo simulado da planta, os valores de $XMEAS_{1-41}$ e XMV_{1-12} , e indicadores calculados como `deriv_norm` e o inventário total de vapor.

3.5 Infraestrutura Local — `tep-supervisor`

O repositório `tep-supervisor` reúne os arquivos de configuração de infraestrutura necessários para executar o laboratório em ambiente local Windows.

3.5.1 Modo de Desenvolvimento — Docker Compose

Para desenvolvimento e testes rápidos, a planta e a IHM são executadas via Docker Compose. O arquivo `local/docker-compose.yml` define:

- Serviço `te-plant`: imagem do simulador, com portas 50051 (gRPC) e variáveis de ambiente de controle (`STEP_DELAY_MS`, `ACTIVE_IDV`).
- Serviço `tep-ihm`: imagem da IHM, com porta 8080 (HTTP/WebSocket), variáveis `GRPC_HOST` apontando para `te-plant:50051`, e volume para persistência dos CSVs.

A rede interna do Docker Compose permite que a IHM alcance a planta pelo nome de serviço `te-plant`, sem necessidade de configuração adicional.

3.5.2 Modo de Produção — Cluster Kind

Para execução do Operator Kubernetes, um cluster local é provisionado com **Kind** (*Kubernetes in Docker*). O Kind executa um cluster Kubernetes completo dentro de contêineres Docker, sem necessidade de máquina virtual separada — adequado para desenvolvimento em Windows.

O roteiro de inicialização do laboratório com o Operator ativo é:

1. Iniciar a planta e a IHM via Docker Compose.
2. Criar o cluster Kind: `kind create cluster -name tep`.

3. Instalar os CRDs no cluster: `kubectl apply -f config/crd/`.
4. Construir e carregar a imagem do operator no cluster: `kind load docker-image tep-operator:latest`.
5. Implantar o operator: `kubectl apply -f config/manager/`.
6. Aplicar a política supervisória: `kubectl apply -f config/samples/`.

A tabela de conectividade entre os componentes em cada modo de execução é resumida na Tabela ??.

Tabela 1 – Endereços de conexão por modo de execução

Componente	Docker Compose	Kind + Operator
IHM → planta	te-plant:50051	te-plant:50051
Operator → planta	N/A	host.docker.internal:50051
Navegador → IHM	localhost:8080	localhost:8080

Fonte: Elaborado pelo autor.

3.6 Considerações Finais

A arquitetura proposta realiza a separação clara entre modelo de planta, camada de controle e interface de operação, com comunicação baseada em contratos gRPC versionados. A `PLCMachine` CRD traduz o conceito de política supervisória do IEC 61499 para o paradigma declarativo do Kubernetes, e o Operator implementa o loop de reconciliação como equivalente ao runtime de implantação do padrão de Operator. O próximo capítulo apresenta os resultados dos experimentos conduzidos com essa arquitetura.

4 RESULTADOS E DISCUSSÕES

Este capítulo apresenta os resultados obtidos em 17 experimentos realizados com a arquitetura descrita no Capítulo ???. Os experimentos são organizados em três fases: (I) inicialização e caracterização do baseline, (II) integração da stack e refatoração arquitetural, e (III) resposta a distúrbios. Todos os experimentos seguem a metodologia Observação → Hipótese → Intervenção → Resultado → Conclusão.

4.1 Metodologia Experimental

Cada experimento parte de uma observação ou questão específica sobre o comportamento do sistema, formula uma hipótese, aplica uma intervenção controlada e registra os resultados. As intervenções foram realizadas localmente via VS Code (configurações por `launch.json` e variáveis de ambiente), sem alterações ao modelo matemático do TEP — preservando a fidelidade ao *benchmark* original (??).

Os dados foram registrados via a funcionalidade `RECORD_CSV` da IHM, gerando arquivos CSV com timestamp, tempo simulado e o vetor completo de XMEAS/XMV a cada passo de amostragem. As análises foram realizadas com a ferramenta `tep_analysis` (Python) com suporte a smoothing via média móvel e geração de gráficos comparativos.

A condição inicial canônica de todos os experimentos de distúrbio é o snapshot `te_exp3_snapshot.toml` gerado no Experimento 3 e validado como ponto de partida estável sem *cold start* no Experimento 4.

4.2 Fase I: Inicialização e Caracterização do Baseline

4.2.1 Experimento 1 — Instrumentação do Startup

O primeiro experimento identificou a causa do *Interlock Shutdown* (ISD) observado durante o *cold start* com `ramp_duration=2,0h`: o reator drenava de 77% para 4,5% enquanto as válvulas de alimentação ainda estavam sendo rampadas. A instrumentação com painéis de *Feed Valve Ramp* e *Purge Flow + Valve* tornou o mecanismo visível: o purge abre em resposta à queda de pressão no início da rampa, mas a realimentação negativa não é a causa principal — o problema dominante é a taxa de reposição de massa insuficiente com rampa longa.

4.2.2 Experimento 2 — Redução do *ramp_duration*

Com `ramp_duration=0,5h`, o nível mínimo do reator durante o *cold start* foi de 59% (margem ampla acima do limite ISD de 10%), e a simulação rodou por mais de 13 h sem ISD. O ponto de operação encontrado — pressão ≈ 2680 kPa, temperatura $\approx 120^\circ\text{C}$, reciclo ≈ 26 kscmh — difere do Modo 1 nominal do FORTRAN (2705 kPa), o que foi

identificado como comportamento esperado dado que os controladores P operam com setpoints fixos.

4.2.3 Experimento 3 — Snapshot de Estado Estável

Removendo IDV(4) e rodando 20 h, a planta convergiu para um attractor com temperaturas de água de resfriamento idênticas aos valores nominais do FORTRAN. O estado final foi salvo como `te_exp3_snapshot.toml` — condição inicial canônica para todos os experimentos subsequentes. A temperatura do reator ($\approx 120^\circ\text{C}$) e a pressão (≈ 2680 kPa) permaneceram estáveis, com drift residual de nível do reator ($\approx +0,6\%/h$) indicando convergência assintótica ainda em curso.

4.2.4 Experimento 4 — Validação do Snapshot

O boot direto a partir do snapshot com `ramp_duration=0,0` foi validado: a planta iniciou diretamente no attractor sem ISD. Observou-se um spike de `deriv_norm` em $t = 0$ (artefato algébrico do RK4 dissipado em $< 0,01$ h), oscilações persistentes no recycle flow ($\pm 0,5$ kscmh) e ausência de deriva no nível do reator — confirmando que o Experimento 3 convergiu para o attractor.

4.2.5 Experimentos 5 e 6 — Investigação das Oscilações e do Balanço de Massa

O Experimento 5 refutou a hipótese de artefato numérico: reduzir `dt` $10\times$ ($0,001 \rightarrow 0,0001$ h) não alterou a amplitude das oscilações do recycle flow, e o `deriv_norm` oscilou em sincronia — evidência de que o vetor de estado em si oscila.

O Experimento 6 adicionou logging dos estados internos UCVR e UCVV. Os holdups foram lisos, descartando oscilações físicas no loop de reciclo. A descoberta principal foi o acúmulo lento ($\approx 0,4\text{--}0,5$ kmol/h) em ambos os vasos — os três controladores P não fecham o balanço de massa.

4.2.6 Experimentos 7, 8 e 9 — Análise do Balanço de Massa

O Experimento 7 mostrou que ajustar o setpoint de pressão de 2705 para 2680 kPa redistribuiu o desbalanço entre UCVR e UCVV mas não o eliminou. O Experimento 8 identificou que todos os oito componentes (A–H) crescem proporcionalmente — composição constante, inventário total crescente — assinatura de desbalanço estrutural, não seletivo. O Experimento 9 testou uma 4ª malha (nível do reator $\rightarrow$ A feed), que piorou o acúmulo ($6\times$ maior): o controlador atuava sobre sintoma, não sobre causa raiz.

A conclusão desta fase: o drift de inventário ($\approx 0,15\%/h$ sobre 682 kmol) é irrelevante para runs de 10–20 h e não compromete os experimentos de distúrbio. O foco dos experimentos seguintes deslocou-se da caracterização do baseline para a avaliação de resposta a distúrbios.

4.2.7 Experimento 10 — Baseline Canônico

O Experimento 10 estabeleceu o baseline de referência quantitativo: 20 h simuladas, sem IDV, com os três controladores P canonicais. As trajetórias de referência são apresentadas na Tabela ??.

Tabela 2 – Trajetórias de referência do baseline (Experimento 10, $t = 0 \rightarrow 20$ h)

Variável	$t = 0$	$t = 20\text{h}$	Drift/h
Pressão do reator (XMEAS ₇)	≈ 2680 kPa	≈ 2700 kPa	+1 kPa/h
Temperatura do reator (XMEAS ₉)	$\approx 120^\circ\text{C}$	$\approx 120^\circ\text{C}$	≈ 0
Nível do reator (XMEAS ₈)	$\approx 69\%$	$\approx 73\%$	+0,2%/h
Nível do separador (XMEAS ₁₂)	$\approx 50\%$	$\approx 50\%$	≈ 0
Nível do stripper (XMEAS ₁₅)	$\approx 50\%$	$\approx 50\%$	≈ 0
Válvula de purge (XMV ₆)	$\approx 39,3\%$	$\approx 39,8\%$	+0,025%/h

Fonte: Elaborado pelo autor a partir dos dados do Experimento 10.

4.3 Fase II: Integração da Stack e Refatoração Arquitetural

4.3.1 Experimento 11 — Desacoplamento dos Controladores

O Experimento 11 refatorou a camada de controle: a lógica dos controladores P foi extraída do `runtime.rs` para a `trait Controller` e o `ControllerBank`. O resultado é uma simulação com CSV numericamente bit-a-bit idêntico ao Experimento 10 — confirmando que a refatoração foi puramente arquitetural, sem efeito sobre a dinâmica. Após esta refatoração, o `runtime.rs` tornou-se agnóstico à lógica de controle, habilitando gestão externa via CRDs.

4.3.2 Experimento 12 — Conectividade do Operator

O Experimento 12 validou a integração completa da stack: planta Rust + Operator Go no cluster Kind + IHM FastAPI. O primeiro obstáculo foi o endereço de conexão: o operator, rodando dentro do Kind, não conseguia alcançar `te-plant.default.svc:50051` porque a planta executa fora do cluster. A solução foi o endereço `host.docker.internal:50051`.

Um segundo obstáculo foi incompatibilidade de versão dos stubs protobuf gerados. Após regeneração dos arquivos `.pb.go` no repositório `tep-operator`, o operator conectou-se com sucesso e passou para a fase `Stable`. Os logs do operator confirmaram:

Listing 4.1 – Log do operator após conectividade estabelecida

```

1 INFO observation cycle complete
2   plantTime: 28019.85 h
3   xmeas_count: 41
4   xmv_count: 12
5   policy_variables: 3
6   deriv_norm: 488.39

```

O status da PLCMachine reportou fase **Stable** com as três variáveis monitoradas dentro da faixa (pressão ≈ 2705 kPa, separador $\approx 50\%$, stripper $\approx 49\%$).

4.3.3 Experimento 13 — Revalidação do Baseline com Nova Infraestrutura

O Experimento 13 verificou que a nova infraestrutura (STEP_DELAY_MS=36, gravação CSV via gRPC) é numericamente equivalente ao Experimento 11. A Tabela ?? compara os resultados.

Tabela 3 – Comparação Experimento 11 vs. Experimento 13 — baseline de referência

Variável	Exp 11	Exp 13	Diferença
Pressão do reator (kPa)	2700,53	2699,56	−0,97 (ruído)
Nível do separador (%)	50,54	49,96	−0,58 (ruído)
Nível do stripper (%)	50,06	50,01	−0,05 (ruído)

Fonte: Elaborado pelo autor a partir dos dados dos Experimentos 11 e 13.

As diferenças são da ordem do ruído de processo, confirmando que a aceleração temporal e o streaming gRPC não introduzem desvios sistemáticos.

4.4 Fase III: Resposta a Distúrbios

4.4.1 Experimento 14 — IDV(4): Temperatura de Entrada do CW do Reator

O Experimento 14 tentou avaliar o efeito de IDV(4), que eleva $+5^\circ\text{C}$ a temperatura de entrada da água de resfriamento do reator. A simulação não atingiu o estado estacionário — colapsou por temperatura no *cold start*, antes que o distúrbio produzisse efeito observável.

A investigação revelou a causa raiz: a implementação *quasi-static* do trocador de calor do reator no Rust produzia $Q_{UR} < 0$ quando T_{CR} caía abaixo de $\approx 67^\circ\text{C}$ durante o arranque, comportamento fisicamente incorreto. A modelagem foi corrigida para $T_{WR} = y[36]$ (fiel ao FORTRAN original, onde YP(37) nunca é atribuído). O distúrbio IDV(4) requer modelagem completa do trocador de calor para ser observável — questão de modelagem fora do escopo deste trabalho.

4.4.2 Experimento 15 — IDV(1): Razão A/C no Feed

IDV(1) aplica um degrau de $-0,03$ mol frac em A e $+0,03$ em C na corrente 4, reduzindo A de 0,485 para 0,455 mol frac. A hipótese era que menos A disponível causaria acúmulo de gás no reator (A é reagente limitante das duas reações principais) com possível novo estado estacionário.

A hipótese foi refutada: a planta não atingiu novo estado estacionário — colapsou por sobrepressão em $t \approx 2,5$ h. Os resultados são apresentados na Tabela ??.

Tabela 4 – Resultados do Experimento 15 — IDV(1), colapso em $t \approx 2,5$ h

Variável	Baseline	Após IDV(1)
XMEAS ₂₃ (A mol%)	$\approx 32\%$	$\downarrow 26\%$ em $t \approx 0,5$ h
XMEAS ₂₅ (C mol%)	$\approx 26\%$	$\uparrow 32\%$ em $t \approx 0,5$ h
XMEAS ₇ (Pressão reator kPa)	≈ 2700 kPa	$\uparrow 2960$ kPa (ISD em 2,5 h)
XMEAS ₉ (Temperatura reator)	$\approx 120^\circ\text{C}$	Flat — sem resposta
XMV ₆ (Purge valve %)	$\approx 39\%$	$\uparrow 62\%$ — insuficiente

Fonte: Elaborado pelo autor a partir dos dados do Experimento 15.

O mecanismo identificado: IDV(1) desacelerou as reações que convertem gás em líquido ($A + C + D \rightarrow G$, $A + C + E \rightarrow H$), causando acúmulo de inventário gasoso no reator com curva de pressão monotonicamente crescente. O controlador P de pressão abriu a purge de 39 para 62%, mas a taxa de remoção não acompanhou o acúmulo. Notavelmente, a temperatura permaneceu absolutamente flat — uma **armadilha diagnóstica**: sem sinal térmico, o operador não tem aviso precoce, e a pressão já está em trajetória de colapso quando se torna observável.

Inserir figura: gráfico do Experimento 15 — XMEAS7 (pressão) e XMV6 (purge) ao longo do tempo, com marcação do ISD.

4.4.3 Experimento 16 — IDV(2): Composição de B no Feed

IDV(2) dobra a fração molar de B na corrente 4 ($0,005 \rightarrow 0,010$ mol frac). B é inerte — não reage — e sua única saída do sistema é a purga. A hipótese era que um novo estado estacionário seria atingido quando a taxa de remoção de B pela purga igualasse a taxa de entrada.

A hipótese foi refutada. O Experimento 16 revelou dois regimes distintos:

1. **Transiente rápido** ($t < 10$ h): pressão sobe ≈ 110 kPa em degrau e parece estabilizar; XMV₆ abre de 42 para 53%. Esse regime induzia a falsa impressão de novo estado estacionário.
2. **Deriva lenta** ($t > 10$ h): pressão continua subindo monotonicamente a $\approx 4\text{--}5$ kPa/h; XMV₆ segue abrindo. O controlador P não tem autoridade suficiente — sem integrador, o erro estacionário cresce com a carga de B.

Os resultados finais são apresentados na Tabela ??.

Inserir figura: gráfico do Experimento 16 — XMEAS7 e XMV6, destacando os dois regimes (transiente rápido e deriva lenta).

Tabela 5 – Resultados do Experimento 16 — IDV(2), colapso em $t \approx 31$ h

Variável	Baseline	Após IDV(2)	Hipótese
XMEAS ₂₄ (B reator mol%)	$\approx 10\%$	$\uparrow 11\%$ e cresce	$\times$ SS estável
XMEAS ₇ (Pressão kPa)	≈ 2727	$\uparrow 2840 \rightarrow$ ISD	$\times$ SS estável
XMEAS ₉ (Temperatura °C)	≈ 120	Flat	confirmada
XMV ₆ (Purge %)	≈ 42	$\uparrow 67\%+$ — insuficiente	$\sim$ parcial

Fonte: Elaborado pelo autor a partir dos dados do Experimento 16.

O contraste com o Experimento 15 é revelador: IDV(1) colapsa em $\approx 2,5$ h por mecanismo cinético (menos A $\rightarrow$ menos consumo de gás); IDV(2) colapsa em dezenas de horas por acúmulo lento de inerte. O ritmo é diferente; o destino é o mesmo. Para o supervisor, IDV(2) representa o caso de *distúrbio mensurável* $\rightarrow$ *resposta insuficiente* $\rightarrow$ *colapso lento* — a lógica supervisória precisa detectar a deriva de pressão antes que o erro acumulado se torne irrecuperável.

4.4.4 Experimento 17 — IDV(3): Temperatura do D Feed

IDV(3) eleva $+5^\circ\text{C}$ a temperatura do feed de D (corrente 2, reagente líquido), alterando a entalpia de entrada. A hipótese era que a temperatura do reator subiria — possivelmente disparando o ISD térmico.

O experimento foi executado por ≈ 1100 h simuladas (≈ 45 dias simulados). Os resultados foram:

Tabela 6 – Resultados do Experimento 17 — IDV(3), 1100 h simuladas

Variável	Baseline	Após 1100 h com IDV(3)
XMEAS ₉ (Temperatura °C)	120,43	120,43 — variação $< 0,01^\circ\text{C}$
XMEAS ₇ (Pressão kPa)	2695	2703 — $+8$ kPa em 45 dias
XMV ₆ (Purge %)	39,1	39,9 — $+0,8$ pp

Fonte: Elaborado pelo autor a partir dos dados do Experimento 17.

A temperatura permaneceu absolutamente flat. A variação de pressão de 8 kPa em 1100 h está dentro do ruído de processo. A análise do código confirmou o mecanismo: a entalpia extra do D feed entra no cálculo de `yp[35]` (UCVV), diluída pelo reciclo dominante (≈ 26 kscmh) antes de atingir o reator. Com `VRNG[0]=400` (D feed range) frente ao reciclo, o impacto térmico é imperceptível.

Conclusão: IDV(3) é numericamente nulo nesta planta e não é um distúrbio útil para exercitar o supervisor. O foco das análises permanece em IDV(1) e IDV(2).

4.5 Análise Integrada e Discussão

4.5.1 Caracterização dos Distúrbios

Os experimentos da Fase III permitiram caracterizar três regimes de distúrbio com implicações distintas para a lógica supervisória:

1. **Colapso cinético rápido — IDV(1)**: a planta colapsa em $\approx 2,5$ h por acúmulo de gás decorrente da desaceleração cinética. O único sinal disponível é a pressão. A temperatura permanece flat, eliminando diagnóstico térmico.
2. **Acúmulo lento de inerte — IDV(2)**: a planta sobrevive o transiente inicial mas entra em deriva monotônica. O tempo de colapso é da ordem de dezenas de horas, oferecendo uma janela de intervenção — contanto que a lógica supervisória detecte a deriva precocemente.
3. **Distúrbio ineficaz — IDV(3)**: nenhuma resposta mensurável em qualquer horizonte prático. Irrelevante para a avaliação do supervisor.

4.5.2 Eficácia dos Controladores P

Os três controladores P do baseline são suficientes para manter a planta estável sem distúrbios por horizonte indeterminado (validado por 20 h no Experimento 10 e implicitamente por 30.000 h no Experimento 12). Entretanto, são insuficientes para rejeitar IDV(1) ou IDV(2): em ambos os casos, a purge abre ao máximo sem conseguir conter o acúmulo.

Essa insuficiência motiva diretamente a camada supervisória: o Operator Kubernetes deve detectar a tendência de pressão crescente e agir — por exemplo, ajustando o setpoint de composição de feed ou reduzindo a carga de B — antes que o erro acumulado torne o colapso inevitável.

4.5.3 Viabilidade do Operator Kubernetes

O Experimento 12 demonstrou a viabilidade técnica do Operator: conexão gRPC estável com a planta via `host.docker.internal`, ciclos de observação contínuos com leitura de todas as 41 XMEAS e 12 XMV, e atualização correta do status do CRD. A fase **Stable** foi mantida por toda a duração do experimento, sem intervenção manual.

A latência de reconciliação (intervalo entre ciclos de observação) ficou em escala de segundos, adequada para a dinâmica do TEP (constante de tempo da ordem de horas). Para aplicações com dinâmica mais rápida, seria necessário ajustar o período de reconciliação ou usar o **StreamMetrics** diretamente no controller.

4.5.4 Contribuição dos 17 Experimentos

A Tabela ?? sintetiza os 17 experimentos, seus objetivos e contribuições principais.

Tabela 7 – Resumo dos experimentos realizados

Exp.	Objetivo	Principal contribuição
1	Instrumentação do startup	Mecanismo de ISD por cold start identificado
2	Redução da rampa de feed	ramp_duration=0,5h estabiliza o startup
3	Snapshot de estado estável	te_exp3_snapshot.toml — base de todos os exps
4	Validação do snapshot	Boot direto seguro; oscilações no reciclo descobertas
5	Investigação das oscilações	Oscilações não são artefato numérico do RK4
6	Componentes de UCVR	Desbalanço de massa global identificado
7	Ajuste de setpoint de pressão	Não elimina desbalanço — redistribui
8	Componente acumulante	Todos crescem proporcionalmente — desbalanço est
9	4ª malha de controle	Malha nível → A feed piora o acúmulo
10	Baseline canônico	Trajetórias de referência para todos os IDVs
11	Desacoplamento dos controladores	ControllerBank — separação modelo/controle
12	Integração do Operator	Operator conectado; fase Stable confirmada
13	Revalidação com nova infraestrutura	STEP_DELAY_MS e gRPC CSV equivalentes ao l
14	IDV(4) — CW do reator	Encerrado; bug de modelagem do trocador corrigido
15	IDV(1) — razão A/C	Colapso cinético em 2,5h; temperatura é armadilha
16	IDV(2) — B no feed	Dois regimes; colapso lento por acúmulo de inerte
17	IDV(3) — temperatura D feed	Numericamente nulo; irrelevante para supervisor

Fonte: Elaborado pelo autor.

4.6 Considerações Finais

Os resultados demonstram que: (a) a planta TEP em Rust replica fielmente o comportamento do modelo original de Downs e Vogel; (b) o Operator Kubernetes é capaz de observar e supervisionar a planta via gRPC de forma contínua e autônoma; (c) os controladores P do baseline são insuficientes para rejeitar os distúrbios IDV(1) e IDV(2) sem lógica supervisória adicional; e (d) IDV(1) e IDV(2) representam casos de interesse para desenvolvimento de estratégias supervisórias, enquanto IDV(3) é ineficaz. O próximo capítulo apresenta as conclusões e perspectivas de trabalhos futuros.

5 CONCLUSÕES FINAIS E TRABALHOS FUTUROS

5.1 Conclusões

Este trabalho investigou a viabilidade de utilizar o Kubernetes — especificamente o padrão de Operator e o mecanismo de Definições de Recursos Customizados (CRDs) — como plataforma de orquestração para aplicações de controle distribuído modeladas segundo os princípios do IEC 61499. A prova de conceito foi realizada sobre uma implementação do Processo Tennessee Eastman (TEP) em Rust, exposta via gRPC, com supervisão Kubernetes em Go/Kubebuilder e interface de operação em FastAPI/WebSocket.

Os principais resultados obtidos permitem afirmar:

1. Viabilidade técnica da orquestração Kubernetes para controle industrial.

O Operator Kubernetes demonstrou capacidade de observar continuamente o estado de uma planta industrial simulada, avaliar variáveis de processo contra uma política declarativa e aplicar ações corretivas via gRPC — tudo de forma autônoma e sem intervenção humana. A fase **Stable** foi mantida por horizonte indeterminado (validado em mais de 30.000 h simuladas), e a latência de reconciliação (escala de segundos) é compatível com a dinâmica lenta do TEP.

2. O CRD **PLCMachine** como equivalente declarativo do modelo de dispositivo IEC 61499.

A separação entre *spec* (política desejada) e *status* (estado observado) do CRD espelha a separação entre modelo de aplicação e estado de execução do IEC 61499. O Operator assume o papel do runtime de implantação e reconfiguração do padrão, tornando o Kubernetes uma infraestrutura viável para o gerenciamento de aplicações de controle distribuído.

3. A separação entre modelo de planta e lógica de controle habilita gestão externa.

A refatoração do Experimento 11, que extraiu os controladores para a trait **Controller** e o **ControllerBank**, foi o passo arquitetural que tornou possível a gestão dos controladores pelo Operator. Esse resultado reforça a importância da separação de responsabilidades em sistemas de controle distribuído — princípio central do IEC 61499.

4. Controladores P são insuficientes para rejeitar IDV(1) e IDV(2).

Os Experimentos 15 e 16 demonstraram que os três controladores P do baseline, embora suficientes para estabilidade nominal, não possuem autoridade de controle suficiente para rejeitar distúrbios composicionais. IDV(1) colapsa a planta em $\approx 2,5$ h por mecanismo cinético; IDV(2) causa colapso lento (≈ 30 h) por acúmulo de inerte. Ambos os casos demandam lógica supervisória de nível mais alto — exatamente o papel do Operator.

5. **IDV(3) é numericamente ineficaz nesta implementação.** O Experimento 17 confirmou que a perturbação de entalpia do D feed é diluída pelo reciclo antes de atingir o reator, não gerando resposta mensurável. Esse resultado é consistente com a análise do código FORTRAN original e delimita o espaço de distúrbios relevantes para validação do supervisor a IDV(1) e IDV(2).
6. **A arquitetura é extensível e substituível.** A comunicação baseada em contratos gRPC tipados (`.proto`) garante que qualquer componente pode ser substituído — incluindo o simulador Rust por dados de uma planta física real — sem alteração dos demais. Isso valida a arquitetura como protótipo realista para cenários industriais.

Em síntese, o trabalho demonstra que o Kubernetes, com seu modelo declarativo e o padrão de Operator, constitui uma plataforma tecnicamente viável para orquestração de aplicações de controle distribuído compatíveis com os princípios do IEC 61499 — respondendo afirmativamente à questão de pesquisa proposta.

5.2 Trabalhos Futuros

A partir dos resultados e limitações identificadas, destacam-se as seguintes direções para trabalhos futuros:

1. **Lógica supervisória de rejeição de IDV(1) e IDV(2).** O próximo passo natural é implementar no Operator a lógica que detecta a trajetória de pressão crescente e age preventivamente — por exemplo, ajustando o setpoint do feed de A ou reduzindo a vazão de B — antes que o acúmulo se torne irreversível. Isso transformaria o Operator de um observador passivo em um supervisor ativo.
2. **Controladores avançados: PID, MPC e aprendizado por reforço.** A trait `Controller` está preparada para receber implementações mais sofisticadas. Controladores PID (com ação integral para eliminar o erro estacionário em IDV(2)) e controladores preditivos baseados em modelo (MPC) são candidatos naturais. A gestão dos parâmetros via CRD permite trocar estratégias de controle em tempo de execução sem parar a simulação.
3. **Horizontal Pod Autoscaler (HPA) para escalonamento de carga computacional.** A arquitetura atual executa a simulação em um único contêiner. Para cenários com múltiplas instâncias de plantas ou experimentos paralelos, o HPA do Kubernetes poderia escalonar automaticamente o número de réplicas com base em métricas de carga da CPU ou tamanho da fila gRPC.

4. **Kubernetes na borda (*edge*) com KubeEdge ou k3s.** Para aplicação em cenários industriais reais, o Operator precisaria executar próximo ao hardware de campo. Distribuições leves do Kubernetes como k3s ou frameworks como KubeEdge (??) são adequados para hardware embarcado industrial, permitindo orquestração de contêineres diretamente em PLCs ou gateways de borda.
5. **Conformidade formal com IEC 61499.** A correspondência identificada neste trabalho entre o CRD `PLCMachine` e o modelo de dispositivo do IEC 61499 é conceitual. Um trabalho futuro poderia formalizar essa correspondência, possivelmente gerando manifests Kubernetes a partir de modelos IEC 61499 criados em ferramentas de engenharia como 4DIAC-IDE.
6. **Modelagem completa do trocador de calor do reator.** O Experimento 14 identificou que a dinâmica do sistema de resfriamento do reator não está completamente modelada na implementação atual (IDV(4) ineficaz). Uma modelagem correta do trocador de calor abriria acesso a mais distúrbios do benchmark e permitiria avaliar o supervisor em cenários de degradação térmica.
7. **Validação com dados de planta real.** A substituição do simulador Rust por um adaptador que consome dados de um sistema SCADA real validaria a arquitetura em condições industriais autênticas. A interface gRPC garante que essa substituição não requer alterações no Operator ou na IHM.

APÊNDICES